SoC Audio Processing

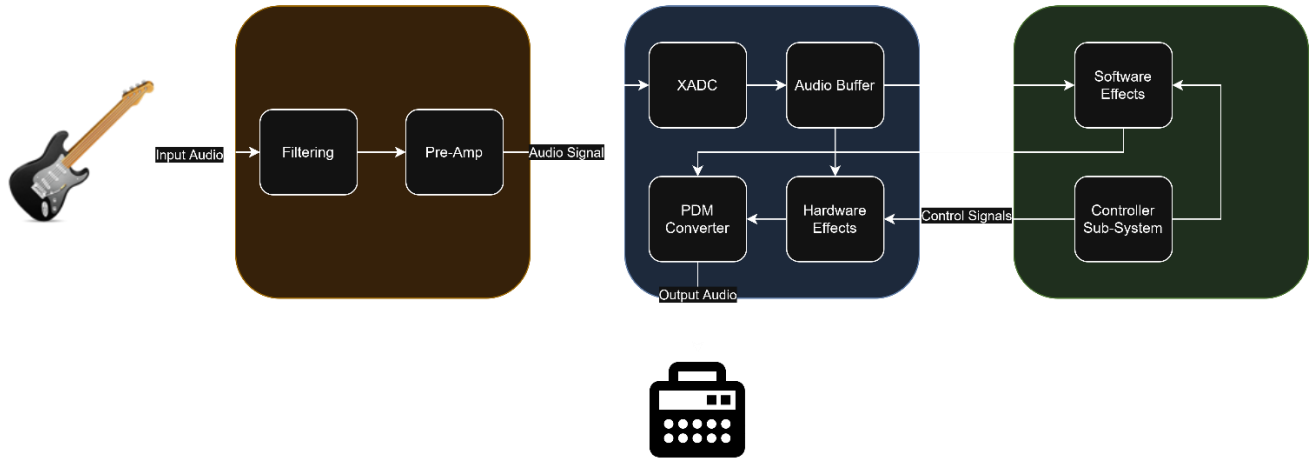
Final Project Report

EEL4930/5934

Case Zumbrum | Cameron Doig-Daniels | Jacob Mack | William Jones

Overview

This project implements a real-time guitar effects pedal on a Spartan-7 FPGA development board (the Digilent Urbana board, part xc7s50csga324-1). An analog audio signal enters through the on-board XADC, passes through one of several configurable effects, and exits as a PWM signal routed to the speaker pins. The system is organized into four main subsystems: the XADC audio input, the processor hardware and AXI fabric, the FPGA hardware effects, and the Linux-based software control layer.



1. Guitar Effects Overview

The system implements five distinct audio effects, split between hardware acceleration on the FPGA fabric and pure software processing on the MicroBlaze CPU. Each effect targets a different aspect of how a guitar signal sounds, ranging from subtle time-based depth to aggressive tonal distortion.

Passthrough (No Effect)

Passthrough is the baseline mode. The raw ADC sample is transferred directly to the PWM output via DMA with no modification. It serves as a reference for verifying signal integrity and latency through the system, and is the mode used when only the FPGA signal path needs to be evaluated without any DSP in the loop.

Echo

Echo is a time-based effect that produces one or more distinct repeats of the original signal. In this implementation, the incoming sample is stored in a circular buffer and retrieved exactly one second later. The delayed copy is attenuated to half amplitude before being mixed with the current sample, so the original note plays at full volume and is followed by a quieter copy one second afterward.

Reverb

Reverb simulates the natural acoustic reflections of a physical space. Where echo produces a few discrete repeats, reverb produces a dense wash of many closely spaced reflections that gradually fade out. This implementation achieves that using a short 100 ms delay combined with a feedback path: each write to the buffer includes a fraction of the value already stored there, so old reflections accumulate and decay over time rather than disappearing after a single repeat.

Hard Clipping (Distortion)

Hard clipping is one of the most common forms of guitar distortion. The effect works by setting a maximum amplitude threshold and clamping any sample that exceeds it to that ceiling. When a guitar signal is loud enough to hit the threshold repeatedly, the tops and bottoms of the waveform are flattened, adding harmonics that give the tone a crunchy, edgy quality.

In this implementation the threshold is set to 1500 out of the full 12-bit range of 4095, which is a fairly aggressive clip level. Hard clipping sounds sharper and more abrasive than soft clipping because the transition from the natural waveform to the clipped ceiling is instantaneous rather than gradual. This is the character found in many overdrive and distortion pedals.

Fuzz

Fuzz is a more extreme form of distortion. The signal is clipped at a much lower threshold (500) so that almost the entire waveform is flattened, and the result is then amplified by a factor of three. This

pushes the output toward a square wave, where the signal spends most of its time at either the maximum or minimum value with very brief transitions in between.

---

2. XADC Audio Input System

The board's built-in XADC IP core (instantiated as `xadc_wiz_0` in the block design) samples the incoming analog signal at 48 kHz through auxiliary differential pair vauxp1/vauxn1, which are pinned out in the XDC file to LVCMOS33 pins B15 and A15. The IP is configured in continuous sequencer mode with channel VAUXP1/VAUXN1 enabled, so samples are always available without any trigger from software.

In the baremetal firmware, the ADC driver is configured with a few key calls after putting the sequencer into safe mode for reconfiguration.

```
XSysMon_SetSeqChEnables(ADC_inst, XSM_SEQ_CH_AUX01);
XSysMon_SetSequencerMode(ADC_inst, XSM_SEQ_MODE_CONTINPASS);
XSysMon_SetAvg(ADC_inst, XSM_AVG_64_SAMPLES);
```

The 64-sample hardware averaging reduces noise from the switching fabric before any effects processing takes place. In the Linux application, the sampled value is accessed through a fixed AXI-Lite offset computed as `0x200 + (17 << 2)` from the XADC base address, which resolves to offset 0x244. The 17 here is a DWord index from the XADC status register base at 0x200, not an ADC channel number. That address is the result register for auxiliary channel 1 (VAUXP1/VAUXN1), which is the physical audio input. The block design assigns the XADC peripheral to base address `0x44A00000`, matching the define in `audio_application.c`.

3. Processor Hardware and AXI Fabric

MicroBlaze Processor

The processor subsystem is built around a MicroBlaze soft-core CPU running at 100 MHz, generated from the 100 MHz on-board oscillator via a MMCM clock wizard (`clk_wiz_0`). The same clock wizard produces 200 MHz and 333 MHz outputs to supply the DDR3 MIG controller, as well as 25 MHz and 125 MHz clocks for other peripherals. The MicroBlaze is configured with a full MMU (required for Linux), hardware multiply and divide units, an FPU, and separate 64 KB instruction and data caches backed by the DDR3 SDRAM.

The MicroBlaze boots from small local LMB (Local Memory Bus) BRAM and then hands off execution to code loaded into DDR3, which is mapped starting at `0x80000000`. A MIG 7-series DDR3 controller drives the on-board MT41K64M16 memory component and exposes an AXI slave interface into the SmartConnect fabric.

AXI Peripheral Interconnect

All audio peripherals connect through a SmartConnect crossbar (`smartconnect_1`) with 8 master ports. The address assignments made in the TCL script correspond directly to the base addresses defined in the Linux application. The PWM GPIO sits at `0x40000000`, the echo GPIO at `0x40010000`, the reverb GPIO at `0x40020000`, the XADC at `0x44A00000`, and the AXI CDMA at `0x44A10000`.

Each effect module interfaces to the processor through a bidirectional AXI GPIO configured at 12 bits wide, where writing a sample to the GPIO output drives the hardware module's `sample_in` port and reading the GPIO input retrieves the processed result from `sample_out`. The AXI CDMA (`audio_cdma`) is instantiated in simple mode with no scatter-gather, which allows the passthrough path to move samples from the XADC register directly into the PWM GPIO register without any CPU involvement after the initial configuration.

4. FPGA Hardware Effects

Clock Divider

Both effect modules share a common clock divider written in SystemVerilog that generates a 48 kHz sample-rate enable pulse from the 100 MHz system clock. Rather than a simple counter, it uses a fractional accumulator approach.

```
acc <= acc + SAMPLE_RATE;
if (acc >= CLK_FREQ) begin
    acc <= acc - CLK_FREQ;
    out <= 1;
end else begin
    out <= 0;
end
```

With SAMPLE_RATE = 48000 and CLK_FREQ = 100000000, the accumulator overflows at the correct average rate with very low jitter. A drifting sample clock causes audible pitch artifacts, so the fractional accumulator is preferable to a plain integer divider here.

Echo

The echo module maintains a 48000-sample circular buffer, giving exactly one second of delay at 48 kHz. On each tick of the sample clock, the module reads the delayed sample, writes the current input, and advances both read and write pointers with wrapping logic. The output mixes the current input with a half-amplitude version of the delayed sample.

```
sum = sample_in + (delayed_sample >>> 1);
```

Saturating guard logic clamps the result to the 12-bit signed range before it is written to `sample_out`, preventing wrap-around distortion if the sum overflows the signed boundary.

Reverb

The reverb module uses the same circular buffer structure but with a 4800-sample delay (100 ms at 48 kHz) and a recursive feedback path. The key difference from the echo module is in the buffer write.

```
buffer[wr_ptr] <= sample_in + (buffer[rd_ptr] >>> 3);
```

This line folds a small fraction of the current buffer contents back into each new write, so reflections build on top of earlier reflections and gradually die down. The output is computed as `sample_in + (delayed_sample >>> 3)`, attenuating the delayed component to one-eighth amplitude. This is a gentler blend than the echo module which uses one-half attenuation (`>>> 1`). The same 12-bit saturation guard from the echo module is applied to prevent overflow. The reverb delay is intentionally much shorter than the echo delay because the recursive feedback means the effect accumulates over many passes rather than producing a single clean repeat.

Verilog Wrappers and Block Design Integration

Both effect modules are wrapped in plain Verilog wrapper files (`echo_wrapper.v` and `reverb_wrapper.v`) that expose unsigned ports. This was necessary because the AXI GPIO IP in Vivado expects standard Verilog-compatible port types, while the inner modules use SystemVerilog signed logic types. The wrappers simply instantiate the inner module and pass signals through, allowing Vivado to treat them as black-box module references in the block design.

The TCL script wires each wrapper directly into its paired AXI GPIO: the GPIO output bus drives `sample_in` and the GPIO input bus reads `sample_out`. This means the processor writes a sample value to the GPIO data register, the effect module processes it, and the processor reads the result back on the same bus in a single read-after-write operation.

PWM Output

The PWM modulator is written in VHDL and converts a 12-bit unsigned duty cycle value into a single-bit output stream. It maintains an internal counter that increments each system clock cycle and sets the output high whenever the counter is below the duty cycle value.

```
pwm_cnt <= pwm_cnt + 1;
pwm_out <= '0';
if pwm_cnt < duty_cycle then
    pwm_out <= '1';
end if;
```

At 12 bits running from a 100 MHz clock, the PWM carrier frequency is approximately 24 kHz, just above the audible range. The block design routes this output to both SPKL (B13) and SPKR (B14) on the XDC, and also to one of the on-board LEDs (LED_PWM at C18) for a visual confirmation of

audio activity. On-board passive filtering on the speaker lines converts the PWM stream back to an analog audio signal.

5. Software and OS Layer

PetaLinux Build and Deployment

The OS is built with PetaLinux 2025.1 targeting a MicroBlaze softcore template. After importing the exported XSA hardware description, the build is configured to strip out networking support and the SSH daemon to reduce image size, while enabling Python 3 and the mmap module for scripting. The final image is loaded over JTAG directly into DDR3. The audio application is packaged as a standard PetaLinux apps recipe so it compiles into the root filesystem and is available at the shell as `audio-application`.

Linux User Space Application

The main application uses `mmap` to map each peripheral's physical address into the process's virtual address space through `/dev/mem`, which requires root access. This avoids writing a kernel driver while still giving direct register access from C.

```
void *echo_map = mmap(NULL, MAP_SIZE, PROT_READ | PROT_WRITE,
                      MAP_SHARED, fd, ECHO_BASE_ADDR);
```

All five regions (XADC, PWM GPIO, echo GPIO, reverb GPIO, and CDMA) are mapped at startup. The user selects an effect from a simple CLI menu, and the application branches into one of five polling loops.

Software Effects

Two distortion effects run entirely in software. The clip function constrains the 12-bit input to a configurable threshold, producing hard clipping distortion.

```
uint16_t clip(uint16_t data) {
    int threshold = 1500;
    if (data > threshold) data = threshold;
    if (data < -threshold) data = -threshold;
    return (uint16_t)(data + 2048);
}
```

The fuzz function applies a lower threshold (500) and multiplies the result by three, producing a more aggressive square-wave-like distortion. Both functions add 2048 before returning to shift the audio value into the unsigned range expected by the 12-bit PWM duty cycle input. One implementation detail worth noting: both functions accept a `uint16_t` parameter, which means the lower-bound check `data < -threshold` is always false since an unsigned value can never be negative. In practice only the upper clamp fires, producing one-sided limiting rather than symmetric clipping. Since the raw ADC output is unsigned (0-4095), a sign-extended or centered representation would be needed for the lower clamp to operate. The hardware effects (echo and reverb) are handled by writing the raw ADC sample to the effect GPIO and reading back the processed result in each loop iteration, so the CPU is still in the data path but the DSP computation runs in fabric.

Passthrough via DMA

For the passthrough mode, the CDMA source and destination addresses are set once to point from the XADC channel register to the PWM GPIO data register, and the transfer is re-triggered in a tight loop by writing a new byte length. This keeps the data path entirely in hardware after initial configuration.

```
dma[DMA_SRC_OFFSET / 4] = (uint32_t)(XADC_BASE_ADDR + ADC_OFFSET);
dma[DMA_DST_OFFSET / 4] = (uint32_t)PWM_BASE_ADDR;
while (1) {
    while (!bit_set(dma[DMA_STAT_OFFSET / 4], 1));
    dma[DMA_LEN_OFFSET / 4] = LENGTH;
}
```

Summary

The design demonstrates a practical division of responsibilities between hardware and software. Long-delay effects that require large on-chip memory buffers and sample-accurate timing run in the FPGA fabric. Simple nonlinear distortion effects that need only a few arithmetic operations per sample run in software where thresholds are easy to adjust. The MicroBlaze processor, AXI SmartConnect fabric, and CDMA engine tie both domains together, and PetaLinux provides a comfortable environment for effect selection and future extension.